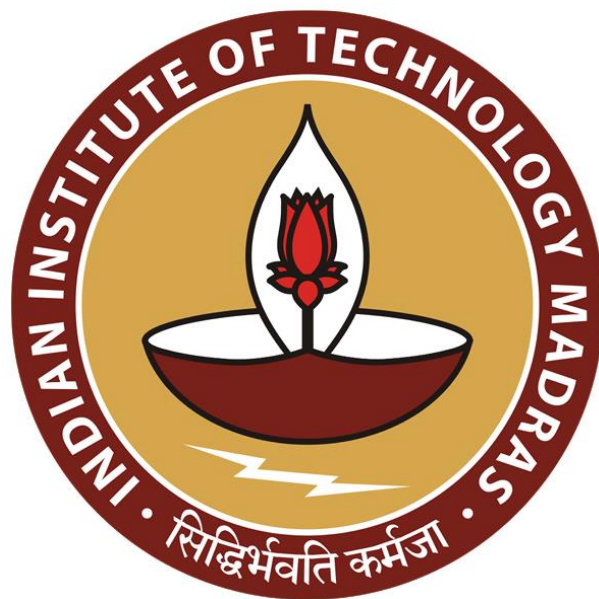




IIT Madras
ONLINE DEGREE

Programming Concepts Using Java
Professor Madhavan Mukund
Department of Computer Science
Chennai Mathematical Institute
Week: 8
Streams

(Refer Slide Time: 00:17)



Operating on collections

- We usually use an iterator to process a collection
 - Suppose we have split a text file as a list of words
 - We want to count the number of long words in the list

```
List<String> words = ....;
long count = 0;
for (String w : words) {
    if (w.length() > 10) {
        count++;
    }
}
```



Madhavan Mukund Streams Programming Concepts using Java 2/9



Operating on collections

- We usually use an iterator to process a collection
 - Suppose we have split a text file as a list of words
 - We want to count the number of long words in the list

```
List<String> words = ....;
long count = 0;
for (String w : words) {
    if (w.length() > 10) {
        count++;
    }
}
```



Madhavan Mukund Streams Programming Concepts using Java 2/9

- We usually use an iterator to process a collection
 - Suppose we have split a text file as a list of words
 - We want to count the number of long words in the list
- An iterator generates all elements from a collection as a sequence
- Alternative approach
 - Generate a **stream** of values from a collection
 - Operations transform input streams to output streams
 - Terminate with a result

```
List<String> words = ....;
long count = 0;
for (String w : words) {
    if (w.length() > 10) {
        count++;
    }
}
```

```
long count = words.stream()
    .filter(w -> w.length() > 10)
    .count();
```



Madhavan Mukund

Streams

Programming Concepts using Java

2/9

So, we have seen collections. So, when we have a collection, we have a lot of items in a single object. And we can run through these using an iterator. So, what is an iterator do? An iterator takes the objects in the collection, and produces them for us one by one as a sequence. So, we take a collection, it could be a list, it could be anything could be a set.

And what the iterator does is it produces the elements in this collection in a sequence one by one? So, for example, supposing we have taken a text file, and we have somehow split it up into a list. So, it is a list of string. So, words, the list of string. And now we want to go through all these words.

And we want to, for example, pick out the words, which are longer than 10 letters and count how many of them there are. So, then we run this iterator. So, we run through every word in this list. And then each time we see a word whose length is bigger than 10, we increment the count. So, this is a standard way of processing an iterator, I mean, processing a collection using an iterator. So, as we said, it generates all these elements in a sequence. So, why not work with the sequence itself. So, an alternative approach with Java provides us is to think of the elements in a collection as a stream. So, imagine that you are watching TV, and you have this some scores or some stock market or something going at the bottom.

So, it is a stream of values, which is going across the screen. So, in the same way, rather than explicitly going through the collection, one element at a time using an iterator, we assume that

the iterator is sort of emitting these values to us as a stream. So, we write some functions which take these streams as input, and process them and produce a stream as output.

So, this is the different style of processing collections using streams. So, here, for instance, just as an example, we take this set of words, which is given to us as a list, we convert it into a stream, then we run a function which filters out. So, it extracts out those words from the stream. So, it is like a stream of objects is going past you on a conveyor belt, and you are pulling out the words that you do not like. So, you are only allowing the words to pass through, which are longer than 10 letters. And then eventually, the words that survived this process will get counted. So, this is how stream processing works at a high level.

(Refer Slide Time: 02:39)

IIT Madras
BSc Degree

Why streams?

- Stream processing is **declarative**
 - Recall, declarative vs imperative
 - Focus on what to compute, rather than how

```
long count = words.stream()
                .filter(w -> w.length() > 10)
                .count();
```

Madhavan Mukund Streams Programming Concepts using Java 3 / 9

Why streams?

- Stream processing is **declarative**
 - Recall, declarative vs imperative
 - Focus on what to compute, rather than how
- Processing can be parallelized
 - `filter()` and `count()` in parallel

```
long count = words.stream()
    .filter(w -> w.length() > 10)
    .count();

long count = words.parallelStream()
    .filter(w -> w.length() > 10)
    .count();
```



Madhavan Mukund

Streams

Programming Concepts using Java 3/9

Why streams?

- Stream processing is **declarative**
 - Recall, declarative vs imperative
 - Focus on what to compute, rather than how
- Processing can be parallelized
 - `filter()` and `count()` in parallel
- Lazy** evaluation is possible
 - Suppose we want first 10 long words
 - Stop generating the stream once we find 10 such words
 - Need not generate the entire stream in advance
 - Can even work, in principle, with infinite streams!

```
long count = words.stream()
    .filter(w -> w.length() > 10)
    .count();

long count = words.parallelStream()
    .filter(w -> w.length() > 10)
    .count();
```



Madhavan Mukund

Streams

Programming Concepts using Java 3/9

So, why do we want to do stream processing rather than iterative processes in `()`(02:45) iterator? Well, remember at the beginning of this course, we talked about the distinction between declarative programming and imperative programming. So, in imperative programming, which is most of the programming that we do in a language like Java, or even Python, we explicitly describe how the computation should be performed. So, we decide which variables we will need to store the intermediate values, which variables will need to store the final values, how we will compute these values and update them, how we will store them, and so on.

So, we gave a very detailed step by step process of actual computation. Whereas in a declarative style of programming, what you are really saying is, I have a set of values, and I tell you some

transformation that I want to achieve. And then assuming this transformation is achieved, I have another set of values, and I tell you what have to do with that, and so on.

So, you just keep describing the effects of the transformation. And to some extent, the implementation of these is not important, or it is something that can be left as a detail to be done separately. So, this is what our stream processing done. So, it basically it says, I do not want to know, in particular, how I am going through the `()`(03:52).

And I am just assuming that I get this stream of objects. And for each of these objects, I tell you, which ones to keep those whose length is greater than or equal to 10. And then finally, I tell you what to do with them, which is to count them. So, this is declarative. So, one of the advantages of doing this kind of stream processing is when I am doing an iterator.

I have to go through the elements one by one, and I have no option because I keep getting the next one. Whereas actually, if I get a stream of elements, then in principle, if the operations that I want to perform on the stream are not dependent on each other, then I can do it in parallel if I have more computational resources at hand.

So, in fact, Java provides a way to create a parallel stream, we will not be looking too much at it right now. But there is one advantage of using streams, which comes from the fact that you can actually deal with the elements in the collection in parallel. The other thing that we can do with a stream is we can do what is called lazy evaluation.

So, normally when I create a sequence of functions. So, I take a function `f`, and I feed the output of `f` as an input to `g`. Then I need to get the entire output of `f` before I can start working with `g`. So, if I have to take a map, for example, in a normal sense, if I map a list to another list, I have to first compute the new list.

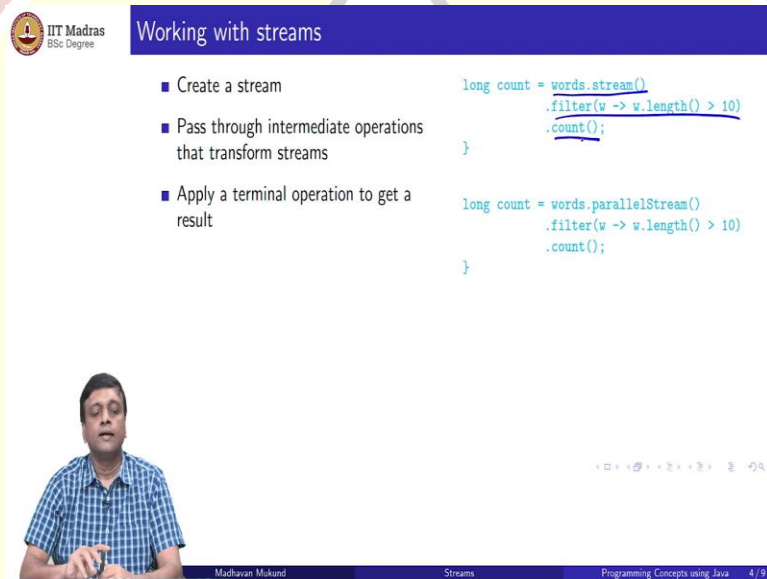
And then I have to take that new list and do something else with that. Now, in lazy evaluation, you start doing the next step as soon as the first step is available. So, for instance, as soon as some words come out of the stream, I can start filtering them. Now, supposing I was not interested in just counting them, but I just wanted the first 10 words in the stream.

Then the moment I have produced 10 words, I can cut off this process. So, I do not need to generate all the stream the entire stream of words, if later on, I get a signal. So, in some sense, I

can pull values from the stream. It is all it is like an iterator in a sense, but it is basically allowing you to think of the stream process as being lazy, it generates a stream a new value whenever it is needs to, and it can stop.

And in particular, this means that if your stream is actually producing an infinite sequence of values, there is no question of stopping that stream, because it is infinite. But if you only need a finite set of values coming out of the stream in the end, then once you have got that, you can stop producing this infinite stream. And we will see examples of where you can actually produce infinite streams in this lecture.

(Refer Slide Time: 06:14)



IIT Madras
BSc Degree

Working with streams

- Create a stream
- Pass through intermediate operations that transform streams
- Apply a terminal operation to get a result

```
long count = words.stream()
                .filter(w -> w.length() > 10)
                .count();

long count = words.parallelStream()
                .filter(w -> w.length() > 10)
                .count();
```

Madhavan Mukund | Streams | Programming Concepts using Java | 4/9

Working with streams

- Create a stream
- Pass through intermediate operations that transform streams
- Apply a terminal operation to get a result
- A stream does not store its elements
 - Elements stored in an underlying collection
 - Or generated by a function, on demand

```
long count = words.stream()
    .filter(w -> w.length() > 10)
    .count();

long count = words.parallelStream()
    .filter(w -> w.length() > 10)
    .count();
```



Madhavan Mukund

Streams

Programming Concepts using Java 4/9

Working with streams

- Create a stream
- Pass through intermediate operations that transform streams
- Apply a terminal operation to get a result
- A stream does not store its elements
 - Elements stored in an underlying collection
 - Or generated by a function, on demand
- Stream operations are non-destructive
 - Input stream is untouched

```
long count = words.stream()
    .filter(w -> w.length() > 10)
    .count();

long count = words.parallelStream()
    .filter(w -> w.length() > 10)
    .count();
```



Madhavan Mukund

Streams

Programming Concepts using Java 4/9

So, to work with streams, we have to do three things, the first thing is we have to create a stream, because if you do not have a stream, you cannot work with it, then we have to apply these transformations, which takes streams as input and produce streams as output. And finally, after doing all the transformations that we are interested in.

We will apply some kind of final operation to get a result. So, we wanted to get for example, in the earlier example, we wanted to count the number of words, which are more than 10 letters long. So, that is a result, which comes after doing some transformation. So, here, for instance, we have this idea that we create the stream, then this operation in the middle is the one that transforms the stream, it drops the words that are too short for us.

And this is the operation that actually computes the result. So, one thing to notice that we could have more than one such here, we have only a single intermediate operation filter, we could have more than one such intermediate operation. So, we really have to think about these streams as passing through. I mean, it is like, it is really passing through a set of operations.

So, it is like, you are applying for a passport. So, you go to some desk, and then they say go to the next desk, and they go to the next desk, so on. But while you are doing that, the next person has also, joined the queue. So, it is not that the entire system waits for you to finish before it admits the next person into that passport queue. Because there are 6 or 7 desks that you have to pass through.

So, at any given point, there are people waiting at each of the desks for their next step. So, in the same way, the stream will also, work that way or no single item actually stores the stream. So, the stream is kind of is a transitory object. So, you should unless you need to, you will not actually keep a copy of the stream in between.

So, the stream is generated at the start, it is processed, and then at the end, it is consumed by the last function, which wants to prepare whatever summary you want, like count or something like that. So, the other thing to note is that the stream itself does not get changed. So, it is not like a function which updates its arguments. So, the input stream is passed to the output stream. And the output stream is a copy with some values transformed or removed or whatever, but the input stream remains as it is. So, the stream processing is not, it does not affect its values, it is not destructive, in a sense.

(Refer Slide Time: 08:39)



Creating streams

- Apply `stream()` to a collection
 - Part of `Collections` interface

```
List<String> wordlist = ...;  
Stream<String> wordstream = wordlist.stream();
```



Madhavan Mukund

Streams

Programming Concepts using Java 5/9



Creating streams

- Apply `stream()` to a collection
 - Part of `Collections` interface
- Use static method `Stream.of()` for arrays

```
List<String> wordlist = ...;  
Stream<String> wordstream = wordlist.stream();  
  
String[] wordarr = ...;  
Stream<String> wordstream = Stream.of(wordarr)
```



Madhavan Mukund

Streams

Programming Concepts using Java 5/9

सिद्धिर्भवति कर्मजा

Creating streams

- Apply `stream()` to a collection
 - Part of `Collections` interface
- Use static method `Stream.of()` for arrays
- Static method `Stream.generate()` generates a stream from a function
 - Provide a function that produces values on demand, with no argument

```
List<String> wordlist = ...;
Stream<String> wordstream = wordlist.stream();
```

```
String[] wordarr = ...;
Stream<String> wordstream = Stream.of(wordarr)
```

```
Stream<String> echos =
    Stream.generate(() -> "Echo");
```

```
Stream<Double> randoms =
    Stream.generate(Math::random);
```



Madhavan Mukund

Streams

Programming Concepts using Java 5/9

Creating streams

- Apply `stream()` to a collection
 - Part of `Collections` interface
- Use static method `Stream.of()` for arrays
- Static method `Stream.generate()` generates a stream from a function
 - Provide a function that produces values on demand, with no argument

```
List<String> wordlist = ...;
Stream<String> wordstream = wordlist.stream();
```

```
String[] wordarr = ...;
Stream<String> wordstream = Stream.of(wordarr)
```

```
Stream<String> echos =
    Stream.generate(() -> "Echo");
```

```
Stream<Double> randoms =
    Stream.generate(Math::random);
```



Madhavan Mukund

Streams

Programming Concepts using Java 5/9

सिद्धिर्भवति कर्मजा

Creating streams

- Apply `stream()` to a collection
 - Part of `Collections` interface
- Use static method `Stream.of()` for arrays
- Static method `Stream.generate()` generates a stream from a function
 - Provide a function that produces values on demand, with no argument
- `Stream.iterate()` — a stream of dependent values
 - Initial value, function to generate the next value from the previous one

```
List<String> wordlist = ...;
Stream<String> wordstream = wordlist.stream();

String[] wordarr = ...;
Stream<String> wordstream = Stream.of(wordarr)

Stream<String> echos =
    Stream.generate(() -> "Echo");

Stream<Double> randoms =
    Stream.generate(Math::random);

Stream<Integer> integers =
    Stream.iterate(0, n -> n+1)
```

$n \rightarrow n+2$



Madhavan Mukund

Streams

Programming Concepts using Java 5/9

Creating streams

- Apply `stream()` to a collection
 - Part of `Collections` interface
- Use static method `Stream.of()` for arrays
- Static method `Stream.generate()` generates a stream from a function
 - Provide a function that produces values on demand, with no argument
- `Stream.iterate()` — a stream of dependent values
 - Initial value, function to generate the next value from the previous one
 - Terminate using a predicate

```
List<String> wordlist = ...;
Stream<String> wordstream = wordlist.stream();

String[] wordarr = ...;
Stream<String> wordstream = Stream.of(wordarr)

Stream<String> echos =
    Stream.generate(() -> "Echo");

Stream<Double> randoms =
    Stream.generate(Math::random);

Stream<Integer> integers =
    Stream.iterate(0, n -> n+1)

Stream<Integer> integers =
    Stream.iterate(0, n -> n < 100, n -> n+1)
```

$0 \rightarrow 1$
 $1 \rightarrow 2$
 $2 \rightarrow 5$
 $n^2 + 1$
 Terminate



Madhavan Mukund

Streams

Programming Concepts using Java 5/9

So, the first step, we said is to create a stream. And we are typically creating streams from collections, that was our motivation that instead of iterating, through a collection, we would like to emit the elements in a collection as a stream of values is a sequence of values. So, quite naturally, there must be a way to do that. And collections, the interface where all the properties of various types of collections were defined, as we saw earlier, actually has a function called stream.

So, we can take any collection, which is defined as part of that hierarchy under the collections interface, and we can apply this function stream to it and it will convert it into a stream and the

type is it is a kind of generic type. So, it is a stream of some underlying value. So, if you start with a collection, like list of string, the underlying value string.

So, if you convert it to a stream, it becomes a stream of strings. So, this is part of the collections interface and this is a standard way that you would convert a collection into a stream. Now, if you had an array now technically arrays are like collections, but they are kind of primitive types in that sense. Although they are objects they are predefined and they are not really sitting in this collections hierarchy. So, you need to do something else. So, there is this function, static function in stream, which is called of, so, the stream.of will take an array and convert it into a stream of the same type.

So, if you have an array of strings, then this stream of() will converted to a stream of strings. Now, the other way to generate a stream is to keep producing the value spontaneously from a function. So, you do not have a collection to start with, you just have a way of generating values on demand. So, a function that will generate values on demand cannot take as anything as an input normally. So, the easiest way to generate such a function is to pass a function which takes no arguments and keeps emitting a value. So, here is one example.

So, there is the stream.generate(). So, what it does is it takes as an argument a function. So, remember these are these lambda expressions. So, this is the input to the lambda expression, this is the output of the lambda expression. So, this is a function which takes no arguments. And if you take no arguments, you have to use this open bracket close bracket to make the lambda expression syntactically valid.

Remember that if you have one argument, then you can omit this if you say v goes to something, then you can omit this. But if you have no arguments, then you cannot just leave a space and write an arrow. So, you have to put this empty bracket. So, this is take no input and produce uniformly the string Echo.

So, the first stream that we are producing is just an infinite stream consisting each element in the stream consisting of the string Echo. The second one is doing something more interesting, it is taking remember the different ways in which we can pass these functions. So, lambda expressions or we can pass the class name and a function in that class.

So, this is a static function, which generates a random number. So, this will now generate these random numbers one after the other. So, they will be different random numbers. But again, in principle, this will go on as long as I want because I have not specified anything. So, these will produce values, remember this lazy thing.

So, whenever I need a new value, in some sense, I will get one more Echo or I will get one more random number. But hypothetically we are producing first and infinite stream of Echoes and an infinite stream of random numbers and then later on processing them. Now, the other way that you might want to generate a stream is to start with a value and then keep producing new values which depend on the previous value, a simple example would be just a kind of iterating sequence like 0 1 2 3.

So, therefore, this is called iterate. So, you start with an initial value, and you provide a function which says, given the previous value, what is a next value. So, if I start with 0, then the next value is 0 plus 1, because n will be 0, and the output will be 0 plus 1. Now, I will say n is 1 and the out so, I could, for example, if I wanted to go through only even numbers, I could say n goes to n plus 2, then I will start with 0, the next one will be 2 and so on.

So, n is not the position n is the previous value generated in this tree. So, what do I do with the previous value to generate the next value. So, this will now generate an infinite sequence of integers 0 1 2 3 the way I have written it, what if you wanted to run this up to a point and stop? Well, you can do that by providing a different format where you provide three arguments.

So, you have the initial value, and you have this function which generates the next value as before, but now, you give a terminating condition, you say that you want to stop if the value that you produce is bigger than 100. Now, notice that this is a condition on the values not a condition of position. Now, it so happens in this particular example, that the values are the positions. But for instance, if I had written n squared, and I use the same condition, then what will happen is that I will go from not n squared, maybe n squared plus 1 might be better.

So, I will go from 0 to n squared plus 1, the next one will be 1, then 1 will produce 1 plus 1 will be 2, 2 will produce 2 squared plus 1 will be 5 and so on and then 5 will produce 25 plus only 26 and then the next value 26 squared will be more than 100. So, this will stop. So, this will only

produce the first four values in that sequence. So, this is not the number of values, but it is a condition on the value is being produced and the moment I reached that condition it will stop.

(Refer Slide Time: 14:17)



Processing streams

- `filter()` to select elements
 - Takes a predicate as argument
 - Filter out the long words

```
List<String> wordlist = ...;  
Stream<String> longwords =  
    wordlist.stream()  
    .filter(w -> w.length() > 10);
```



Madhavan Mukund Streams Programming Concepts using Java 6 / 9



Processing streams

- `filter()` to select elements
 - Takes a predicate as argument
 - Filter out the long words
- `map()` applies a function to each element in the stream.
 - Extract the first letter of each long word

```
List<String> wordlist = ...;  
Stream<String> startlongwords =  
    wordlist.stream()  
    .filter(w -> w.length() > 10)  
    .map(s -> s.substring(0,1));
```



Madhavan Mukund Streams Programming Concepts using Java 6 / 9

Processing streams

- `filter()` to select elements
 - Takes a predicate as argument
 - Filter out the long words
- `map()` applies a function to each element in the stream.
 - Extract the first letter of each long word
- What if `map()` function generates a list?
 - Suppose we have `explode(s)` that returns the list of letters in `s`
 - `map()` produces stream with nested lists

```
List<String> wordlist = ...;
Stream<String> longwords =
    wordlist.stream()
        .filter(w -> w.length() > 10);
```

```
List<String> wordlist = ...;
Stream<String> startlongwords =
    wordlist.stream()
        .filter(w -> w.length() > 10)
        .map(s -> s.substring(0,1));
```

```
List<String> wordlist = ...;
Stream<String> startlongwords =
    wordlist.stream()
        .filter(w -> w.length() > 10)
        .map(s -> explode(s));
```

"list"
↓
"l", "i", "s",
"t"



Madhavan Mukund

Streams

Programming Concepts using Java 6/9

Processing streams

- `filter()` to select elements
 - Takes a predicate as argument
 - Filter out the long words
- `map()` applies a function to each element in the stream.
 - Extract the first letter of each long word
- What if `map()` function generates a list?
 - Suppose we have `explode(s)` that returns the list of letters in `s`
 - `map()` produces stream with nested lists
- `flatMap()` flattens (collapses) nested list into a single stream

```
List<String> wordlist = ...;
Stream<String> longwords =
    wordlist.stream()
        .filter(w -> w.length() > 10);
```

```
List<String> wordlist = ...;
Stream<String> startlongwords =
    wordlist.stream()
        .filter(w -> w.length() > 10)
        .map(s -> s.substring(0,1));
```

```
List<String> wordlist = ...;
Stream<String> startlongwords =
    wordlist.stream()
        .filter(w -> w.length() > 10)
        .flatMap(s -> explode(s));
```



Madhavan Mukund

Streams

Programming Concepts using Java 6/9

So, that was as far as generating streams. So, we can generate a stream from a collection or we can generate a stream using a function. Now, we have generated the stream and we want to process it. So, we already saw the simplest form of processing which is a filter. So, filter will take a predicate. So, what is a predicate a predicate is a function which takes an input and tells you true or false. And what filter does is it throws away all the values for which the predicate returns false it keeps only the true.

So, in our case the predicate was map a word `w` to the predicate that it is length is greater than 10. So, if it is length is greater than 10, the word is mapped to true if it is not greater than 10 it is word mapped to false. So, all the false values, all the words whose length is less than or equal to

10 will be dropped. And this will filter out the short words from your sequence and keep only the long words.

So, filtering, I mean, when you think of filtering it in real life, you think of it both ways you apply a filter, you want to keep something out, and you want to let something through. So, filter, the predicate is actually telling you what let us go through, not what is retained, we are not interested in what is retained here, what is retained as a short word, they get thrown away, what is important is what goes through.

So, that is the filtered output. So, filtering is a very simple process of selecting a sub sequence out of the sequence. But more often, we want to transform the sequence as we go along. And this is called map. So, map applies a function to each element in the stream. So, here, for instance, again, we give a expression which says, so, notice that up to this point, we have filtered out all the long words.

So, words whose length are at least 10, at least 11, rather greater than 10. And now what we are saying is for each of those words. So, this dot by the way is the sequence concatenation operator, it says, pass stream concatenate, pass this stream to this thing to this thing to this thing. So, I will start with this stream, I pass it to filter, and the output filter is now being passed to map.

So, map now has a function which it wants to apply to every element that it gets. And this function is to take the string, remember, this is now a stream of strings. And these are long strings that two strings have at least like the level. So, it takes each string and it maps it to its first letter, it takes a substring starting at position 0 of length 1, which is numbering the first letter. So, now this is extracted the first letter of each long word. So, now it could be that map actually does something more than that.

So, what if instead of extracting the first letter, supposing we had a function, it is not a real function in Java, you would have to write it. But let us suppose that we had a function called explored, which you take a word string, like to take a string like list, and explode it as a stream or a sequence with four letters.

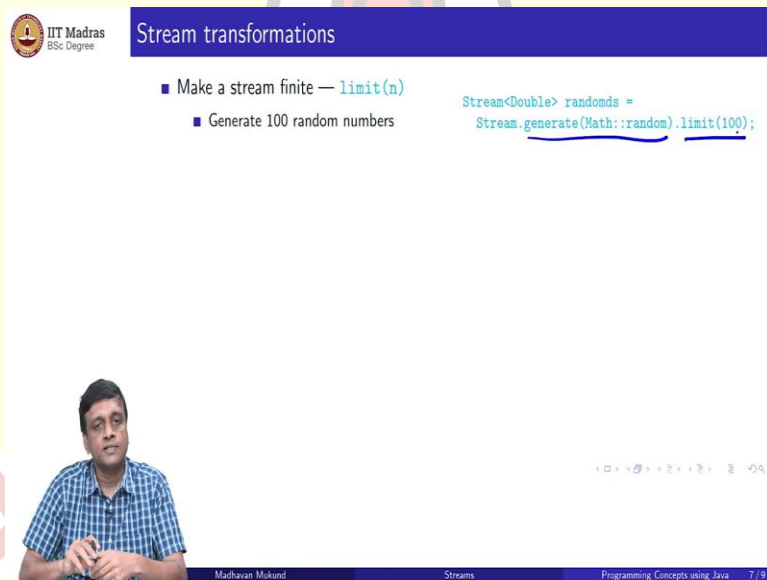
So, now what map is doing is, it is taking one value and making it into a list of values. So, now if I have a sequence of words, a stream of words coming, each word actually becomes a nested list.

So, I have a list and list a list a list. So, I have a stream of lists. So, very often, we do not want a stream of lists, we want to actually think of this whole thing as just one stream.

So, it is just I have replaced every word by its constituent letters. And now I want a sequence of all the letters across all the long words. So, for this, I can now do something which is a variation of map called a flat map. So, earlier, I had said, map each string to explode of s. Now, I am saying flat map each string to explode of s.

So, all this says is that the output of this map is going to be having one next level of brackets, if you want to think of it and dissolve those brackets, it collapses, a list of lists into a list or a stream of list into a single stream. So, that is what flat map has. So, these are some of the things that one can do to process streams.

(Refer Slide Time: 18:22)



IIT Madras
BSc Degree

Stream transformations

- Make a stream finite — `limit(n)`
- Generate 100 random numbers

```
Stream<Double> randoms =  
    Stream.generate(Math::random).limit(100);
```

Madhavan Mukund Streams Programming Concepts using Java 7/9

Stream transformations

- Make a stream finite — `limit(n)`
 - Generate 100 random numbers
- Skip `n` elements — `skip(n)`
 - Discard first 10 random numbers

```
Stream<Double> randoms =  
    Stream.generate(Math::random).limit(100);
```

```
Stream<Double> randoms =  
    Stream.generate(Math::random).skip(10);
```



Stream transformations

- Make a stream finite — `limit(n)`
 - Generate 100 random numbers
- Skip `n` elements — `skip(n)`
 - Discard first 10 random numbers
- Stop when element matches a criterion — `takeWhile()`
 - Stop with number smaller than 0.5

```
Stream<Double> randoms =  
    Stream.generate(Math::random).limit(100);
```

```
Stream<Double> randoms =  
    Stream.generate(Math::random).skip(10);
```

```
Stream<Double> randoms =  
    Stream.generate(Math::random)  
        .takeWhile(n -> n <= 0.5);
```



सिद्धिर्भवति कर्मजा

Stream transformations

- Make a stream finite — `limit(n)`
 - Generate 100 random numbers
- Skip `n` elements — `skip(n)`
 - Discard first 10 random numbers
- Stop when element matches a criterion — `takeWhile()`
 - Stop with number smaller than 0.5
- Start after element matches a criterion — `dropWhile()`
 - Start after number larger than 0.05

```
Stream<Double> randoms =  
    Stream.generate(Math::random).limit(100);
```

```
Stream<Double> randoms =  
    Stream.generate(Math::random).skip(10);
```

```
Stream<Double> randoms =  
    Stream.generate(Math::random)  
        .takeWhile(n -> n >= 0.5);
```

```
Stream<Double> randoms =  
    Stream.generate(Math::random)  
        .dropWhile(n -> n <= 0.05);
```



Madhavan Mukund

Streams

Programming Concepts using Java 7/9

Stream transformations

- Make a stream finite — `limit(n)`
 - Generate 100 random numbers
- Skip `n` elements — `skip(n)`
 - Discard first 10 random numbers
- Stop when element matches a criterion — `takeWhile()`
 - Stop with number smaller than 0.5
- Start after element matches a criterion — `dropWhile()`
 - Start after number larger than 0.05
- Can also combine streams, extract distinct elements, sort, ...

```
Stream<Double> randoms =  
    Stream.generate(Math::random).limit(100);
```

```
Stream<Double> randoms =  
    Stream.generate(Math::random).skip(10);
```

```
Stream<Double> randoms =  
    Stream.generate(Math::random)  
        .takeWhile(n -> n >= 0.5);
```

```
Stream<Double> randoms =  
    Stream.generate(Math::random)  
        .dropWhile(n -> n <= 0.05);
```



Madhavan Mukund

Streams

Programming Concepts using Java 7/9

Now, these are transforming streams, by value, dropping them and so on. So, one of the natural things that you might want to do to a stream is to limit it to a certain length. So, there is a function called `limit`, which takes an integer and it limits it to that many things. So, remember our earlier function `generate`, which was generating this infinite sequence of random numbers generated by `Math.random`.

So, now we can say I do not want an infinite sequence, I only want 100 of them. So, now I take this stream, which is infinite, and I pass it to this `limit`. So, it will stop generating numbers after the first 100. So, this lazy evaluation will take over. So, this will give me the first 100 random numbers produced by that stream.

Or I could say, I do not know if the first few numbers are really random, because these are all pseudo random, they start with some seed and all that. So, let me wait for the thing to become really random. So, skip the first step. So, there is a function called skip, which will take the stream and drop the first few elements in this case 10, whatever number you tell it to skip and then continue from there onward.

So, this will now take an infinite stream, and it will produce an infinite stream, but having dropped some prefix of it. Now, this stopping and starting might be based not on a fixed number, but on some value. We already saw that we had the way of generating using a value saying stop generating when you do this, but whatever stream is coming and we are looking for something and we want to stop and we see that something.

So, take while says keep reading the input until some condition holds. So, in this case, it is saying check whether the number that you see. So, random will generate numbers between 0 and 1. So, what we are saying is whenever I first see a random number which is bigger than half, I will stop.

So, take while says so, long as n does not set. So, is really when this number, take while says so, long as this predicate is true keep so, it will not stop it will stop when you first see a number smaller than 0.5. So, it says take while the predicate is true. So, like any while loop, while condition do something.

So, while this predicate is true, keep accepting things a stream, the moment the predicate becomes false stop. So, long as you are producing numbers bigger than equal point 5, we will continue the moment you see a number which is smaller than point 5 you will stop. So, this is take while and correspondingly just like so, this is like a limit with a condition.

So, the limit is achieved by when condition becoming false. And the skip is with a condition is called dropWhile. So, it says until some condition becomes true, I will drop it. So, long as the numbers are smaller than 0.05. I will ignore them the moment I see a first number which is bigger than 0.05. I will start from there and continue.

So, remember that this is only indicating the termination point. So, take while says the moment I see a number which is smaller than 0.5 stop, even if I see bigger numbers after that. So, it is not a

filter, it is different from filter what filter will do is it will keep on applying these even beyond that take while says the moment the condition becomes false stop, dropWhile says the moment the condition becomes false start.

So, as long as the condition is true, skip after this condition becomes false start and then ignore the condition after, the condition is not operating after that. So, that is the difference between take while and dropWhile and filter. So, filter keeps on going, the condition is true and false at various points here, once the condition switches you stop, either you terminate the stream or you start the stream.

So, you can do other things with streams, you can take two streams and merge them, you can combine them, concatenate them extract distinct elements, we can sort them and so on. So, you can look up the Java documentation, there are quite a few natural things that you can do with streams. And you can look up how to do that.

(Refer Slide Time: 22:17)

Reducing a stream to a result

- Number of elements — `count()`
- Count random numbers larger than 0.1

```
long countrand =  
Stream.generate(Math::random)  
  .limit(100).  
  filter(n -> n > 0.1)  
  .count();
```

~ 10

0 0.1 1

Madhavan Mukund Streams Programming Concepts using Java 8 / 9

Reducing a stream to a result

- Number of elements — `count()`
 - Count random numbers larger than 0.1
- Largest and smallest values seen
 - `max()` and `min()`
 - Requires a comparison function

```
long countrand =
    Stream.generate(Math::random)
        .limit(100)
        .filter(n -> n > 0.1)
        .count();
```

```
Optional<Double> maxrand =
    Stream.generate(Math::random)
        .limit(10)
        .max(Double::compareTo);
```



Madhavan Mukund

Streams

Programming Concepts using Java 8/9

Reducing a stream to a result

- Number of elements — `count()`
 - Count random numbers larger than 0.1
- Largest and smallest values seen
 - `max()` and `min()`
 - Requires a comparison function
 - What happens if the stream is empty?
Return value is optional type — later

```
long countrand =
    Stream.generate(Math::random)
        .limit(100)
        .filter(n -> n > 0.1)
        .count();
```

```
Optional<Double> maxrand =
    Stream.generate(Math::random)
        .limit(100)
        .filter(n -> n < 0.001)
        .max(Double::compareTo);
```



Madhavan Mukund

Streams

Programming Concepts using Java 8/9

सिद्धिर्भवति कर्मजा

- Number of elements — `count()`
 - Count random numbers larger than 0.1
- Largest and smallest values seen
 - `max()` and `min()`
 - Requires a comparison function
 - What happens if the stream is empty?
Return value is *optional* type — later
- First element — `findFirst()`
 - First random number above 0.999
 - Again, deal with empty stream
- And more ...

```
long countrand =
    Stream.generate(Math::random)
        .limit(100)
        .filter(n -> n > 0.1)
        .count();
```

```
Optional<Double> maxrand =
    Stream.generate(Math::random)
        .limit(100)
        .filter(n -> n < 0.001)
        .max(Double::compareTo);
```

```
Optional<Double> firstrand =
    Stream.generate(Math::random)
        .limit(100)
        .filter(n -> n > 0.999)
        .findFirst();
```



Madhavan Mukund

Streams

Programming Concepts using Java 8/9

So, now let us move to the last part of this pipeline. So, we started with creating a stream, then we had these functions, which will transform streams in different ways, filter them, map them, limit them by some condition, and so on. And now we reach at the end something that we want to do with it. So, we want to, in some sense, collapse this whole sequence into some number or some value. So, the thing that we saw earlier was count, we wanted to count the number of long words.

So, here, for instance, if we are looking at the random generation thing, we might want to say that I generate 100 random numbers. And then I filter those, which are bigger than 0.1. So, I am counting now those which survived this filter, those which are passing through this filter. So, how many random numbers in 100? Did I generate which are larger than 0.1. So, since the random numbers are between 0 and 1, and if it is uniformly distributed, you would expect that 10th of them are in the range 0 to 1.1 and 90th of them are in the range, because we have a range from 0 to 1. And you are saying that at 0.1.

So, you are saying how many of them are here. So, if you have filtering out those numbers which are bigger than that, and you have 100 of them, you should expect count to be approximately 90. If this is a good (23:33). So, this is one way to maybe validate whether your random function is working the way you expect or not. So, the other thing that you might want to do is check something like the maximum or the minimum. So, I take say the first 10 random numbers and I asked what is the maximum one.

Now, in general, you can be flexible about how you compare these things. So, you have to pass a function in this case, these numbers are coming from random. So, they are doubles. So, we will pass the static compare to function which is defined in double and this will allow us to compare two doubles in the normal way. So, that the maximum is the largest number, but the important thing is that you must pass a function a comparison function to max and to min. So, otherwise what it will do is it will produce the max.

Now, there is something strange happening here. So, you are taking a stream of doubles, you are finding the maximum. So, therefore, the maximum should be one of those doubles, and therefore the result should be a double. But it is the return value of this is something different from double. So, why is that? Well, the reason is that I could actually have something in between before I take the max for instance.

Supposing I am asking for filtering the first 100 numbers to include only those which are very, very small. Now, it is possible that none of the first 100 numbers meet that predicate none of them are actually smaller than 0.001. So, if none of them are that small, then what will happen is that the effect of this filtering will be to produce an empty string.

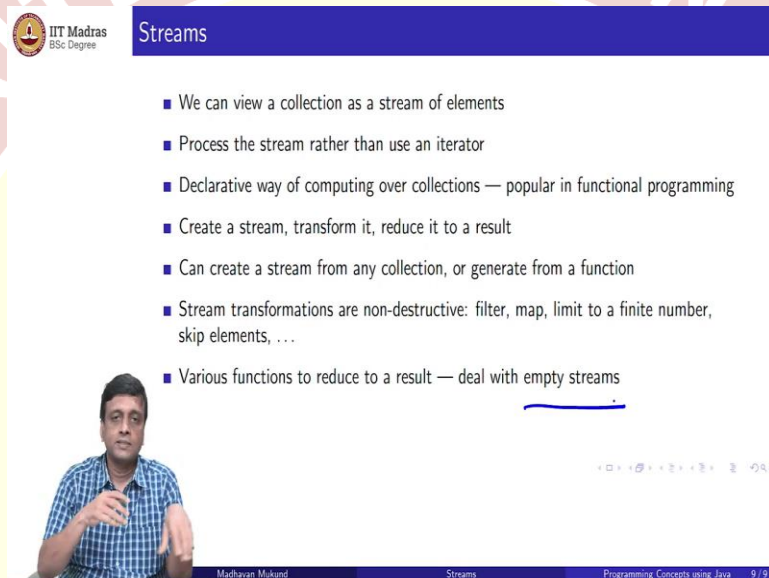
I start with 100 random numbers. And then I am filtering them through this very selective predicate, which looks for very, very small ones. But it is possible that in a random sequence of 100, none of them meet it, and then I produce something which is empty. And now I asked max, to find the maximum of an empty sequence.

So, as we all know, the maximum of an empty sequence is not something that is well defined. So, in this case, max will traditionally would return something like false or null or something like that. And this is the reason why we have this kind of optional thing. So, this says that the outcome of this computation is likely to be a double, but it may not be a double.

And we will discuss these optional types in more detail in the next week. So, another thing that one can do is to find the first element. So, you want to take the first 100 numbers, filter it and find the first one. Now again, the problem is that the filtering might produce an empty sequence. So, the first element is well defined if the sequence or the stream is not empty.

But I have now again, given a given the reverse of this, I have said, supposing it is a very, very random number very close to 1. So, that these numbers are between 0 and 1. So, if it is bigger than point 999 a very small fraction are going to be there, it is possible that nothing survives. So, again, find first may have nothing to find the first off. So, again, this like our earlier, max, and min functions will return not always the type that you expect it will return an optional version of that type. So, we will deal with this later.

(Refer Slide Time: 26:41)



IIT Madras
BSc Degree

Streams

- We can view a collection as a stream of elements
- Process the stream rather than use an iterator
- Declarative way of computing over collections — popular in functional programming
- Create a stream, transform it, reduce it to a result
- Can create a stream from any collection, or generate from a function
- Stream transformations are non-destructive: filter, map, limit to a finite number, skip elements, ...
- Various functions to reduce to a result — deal with empty streams

Madhavan Mukund Streams Programming Concepts using Java 9 / 9

So, what we have seen is that we can take a collection, any type of collection, and where we would normally produce a sequence of values using an iterator, we can instead produce a stream using the `collections.stream` function. So, we can always think of a collection as a stream of values. And then we can process the stream rather than running through it using a conventional iterator. And this is nice, because we can use these declarative kinds of things and get what is actually a functional programming style of working with collections.

So, collections you can actually operate on as streams. So, we can create a stream as we said. So, we can also create a stream be set from a function, and we can write a generate function which generates a stream. So, we create the stream either from a collection or from a function, we pass it through this stream transformation, which are not disruptive.

They take an input stream, produce an output stream, then you concatenate. So, it is like connecting a sequence of pipes. So, you pass this flow through this pipe, it goes through some

mixing box comes out as a stream of values, possibly different color in the next pipe, and so on. And then eventually at the end of this pipeline, you take these things, and you do something to it to produce a result.

And when you produce a result, one of the problems we have to deal with is the empty streams, because in this whole process of mixing and matching and filtering, and all that we might have actually generated something which produces no output. So, if we have no output, then the result that we are trying to compute may not be a valid thing.

If it is a count, for instance, it will just be 0. But if it we saw if it is max, or if it is the first element or the last element or something like that. That does not make sense. So, therefore, we have to be careful about empty streams, and we will see how to deal with them with these optional types later.

